IEEE
CAIRO UNIVERSITY STUDENT BRANCH

Introduction to Digital Electronics

Made By Ahmed Abdellatif & Mahmoud Matar

2024/2025

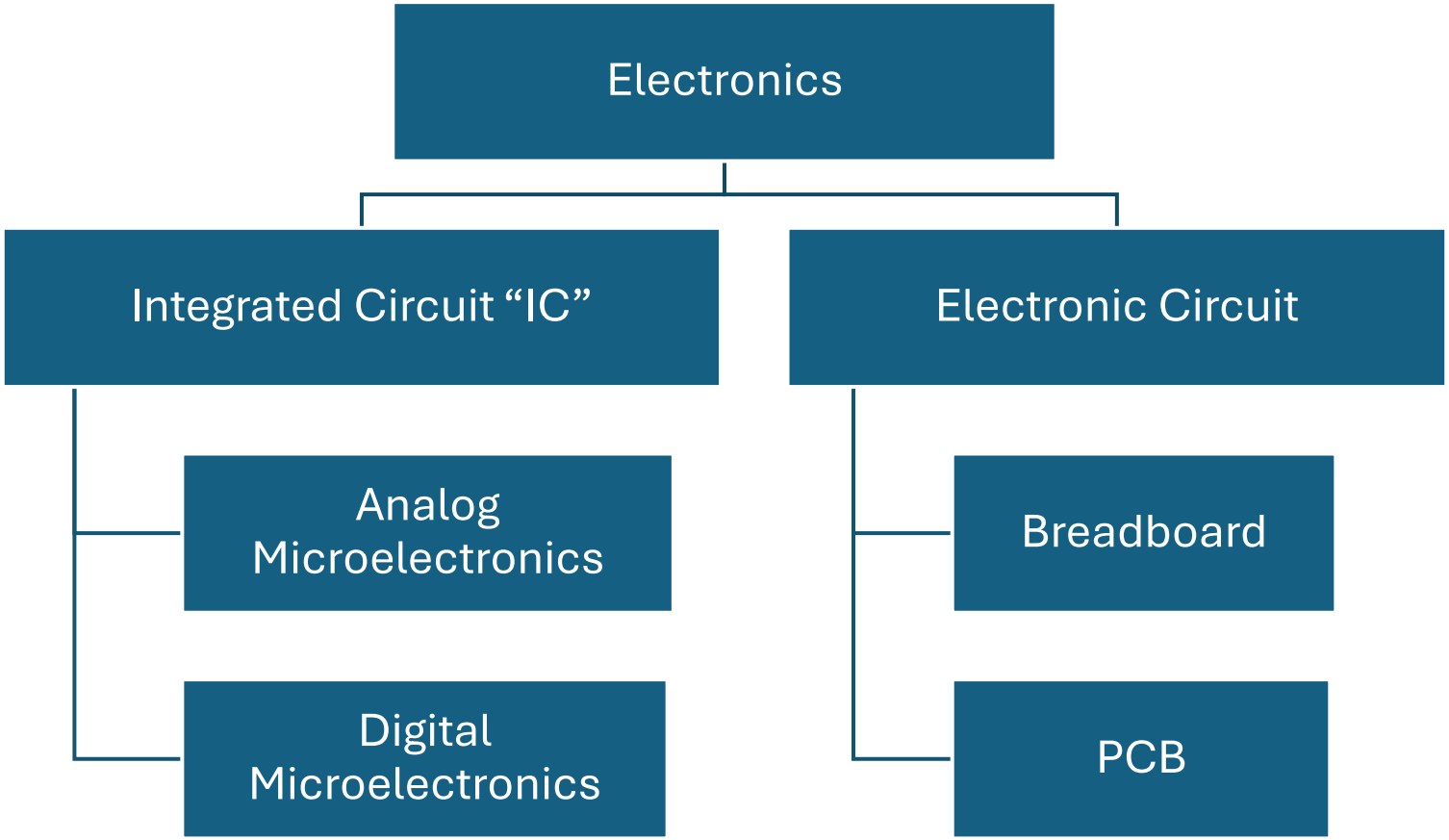
IEEE

Agenda

- What is electronics? And Why IC?
- Abstractions of systems
- Digital VS Analog
- Digital design flow
- FPGA VS ASIC
- IC industry in Egypt
- What is HDL & Verilog
- Verilog Numeric Construct
- Verilog Operators
- Start Coding Using Verilog

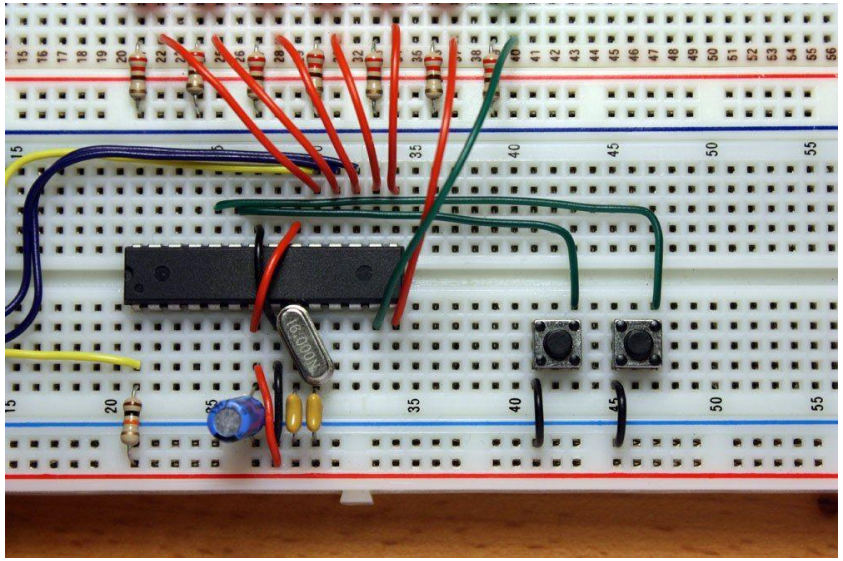
Electronics ??

Electronics Categories

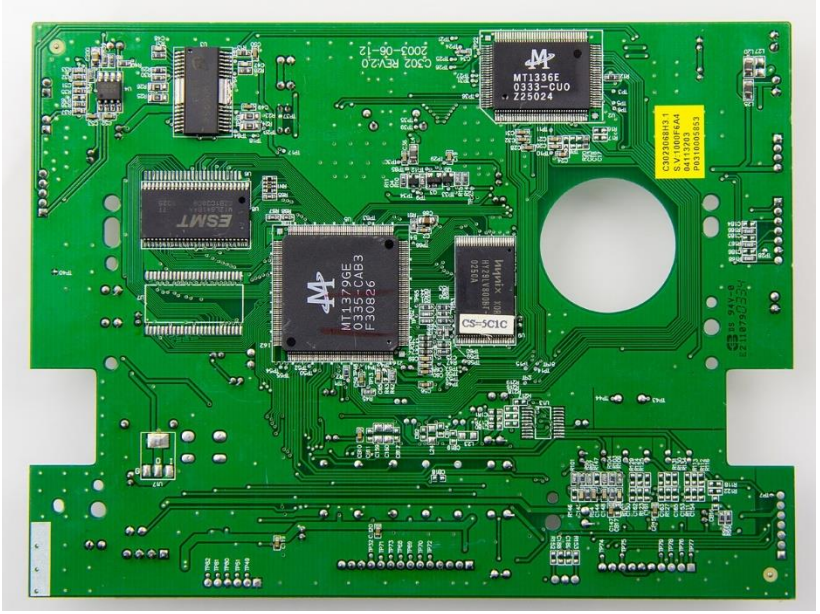


Electronic Circuit

Breadboard

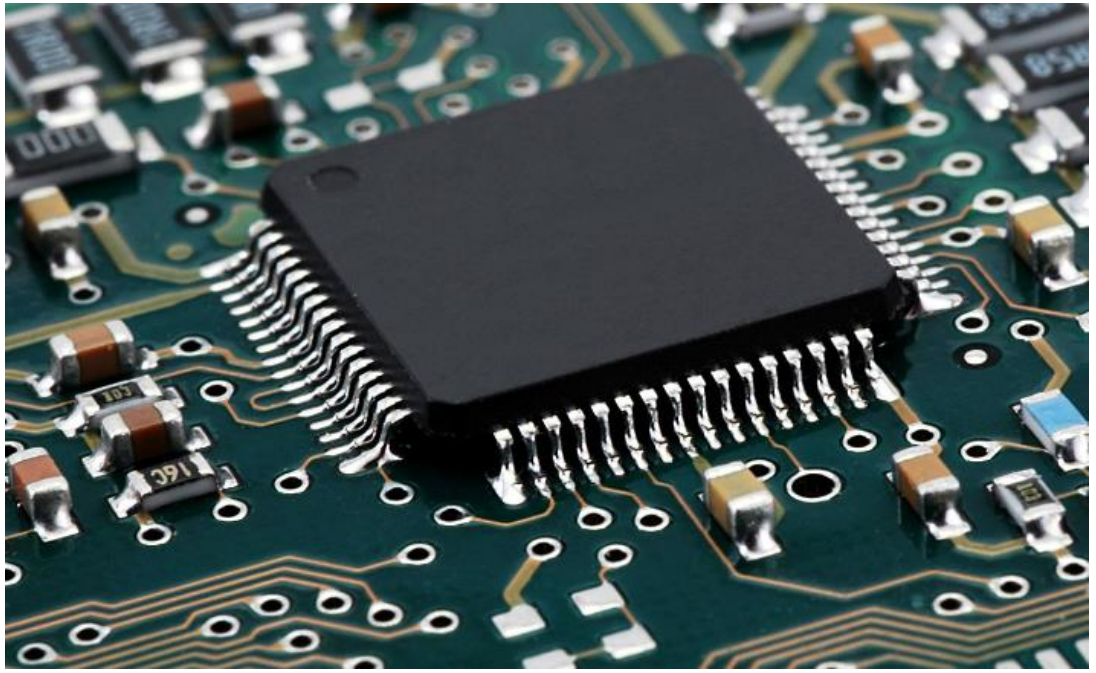


Printed Circuit Board “PCB”



IC

- It’s Smaller version of Electronic System.
- Plays very important role in our life.
- It’s built from Silicon.
- The Building Block of IC is the Transistor

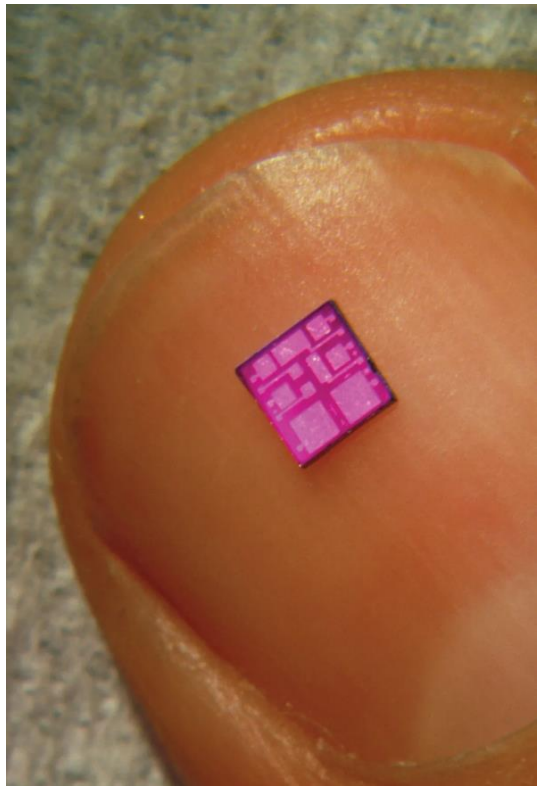


Why IC ??

Benefits of Integrated Circuits

First: Size

- IC can be very very small.
- This Allows for new horizons of Applications.



Benefits of Integrated Circuits

Second: Cost

- With mass Production, the cost of IC Fabrication can be negligible.

Third: Power Consumption

- Since Transistor are very tiny , they flow less current = Less power to do the same role of the Large System.

- This helps us in field of Portable devices Like phones.

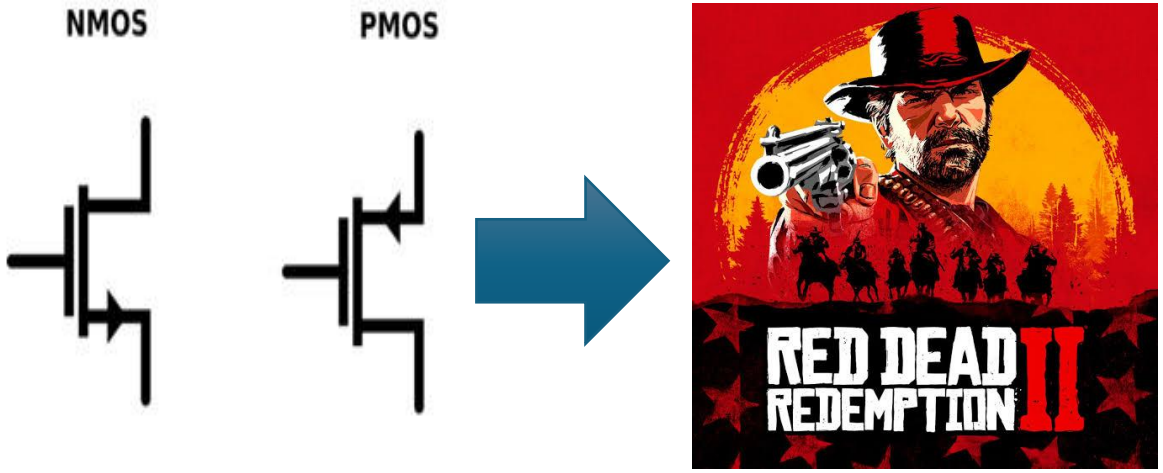
Benefits of Integrated Circuits

Fourth: Performance

- The Performance of Average Processor today is Just impressive. This allow for new fields that requires very high computation power such as AI and Image processing

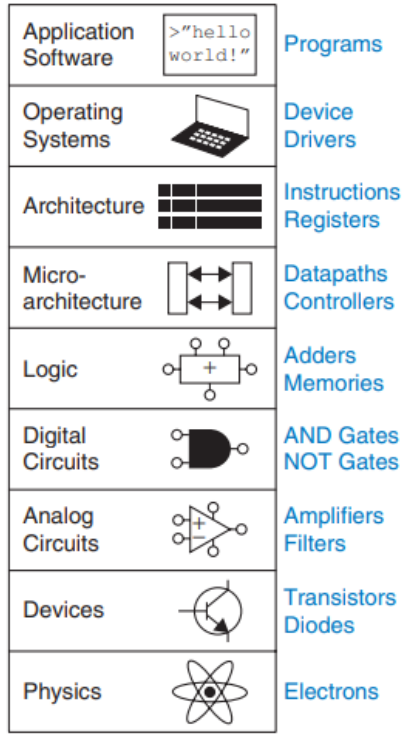
How we design IC to do These Things

- IC plays an essential role in our life today.
- **But how we can design an IC with Millions and Billions of Transistors to do such Amazing Things?**
- The Secret word is “**Abstraction**”



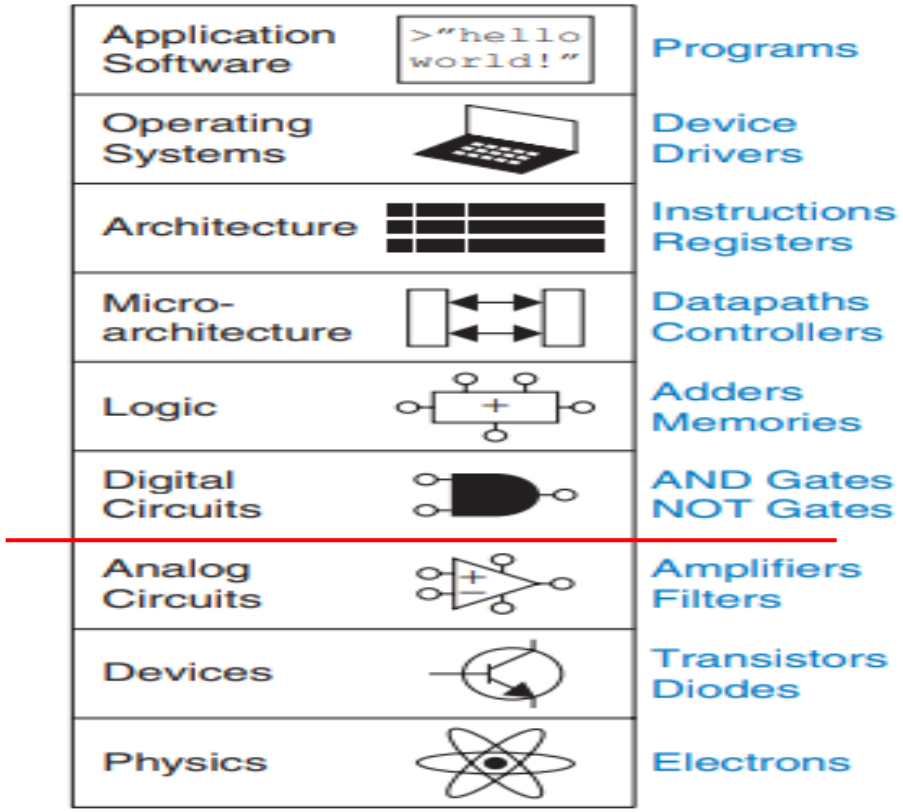
Abstraction in Digital Systems

- We divide any System into Layers of Abstraction.
- Each Layer has its People who devoted their lives to study and work in it.
- But none of them interfere in how the layer beneath him operate “it just operates as it should”



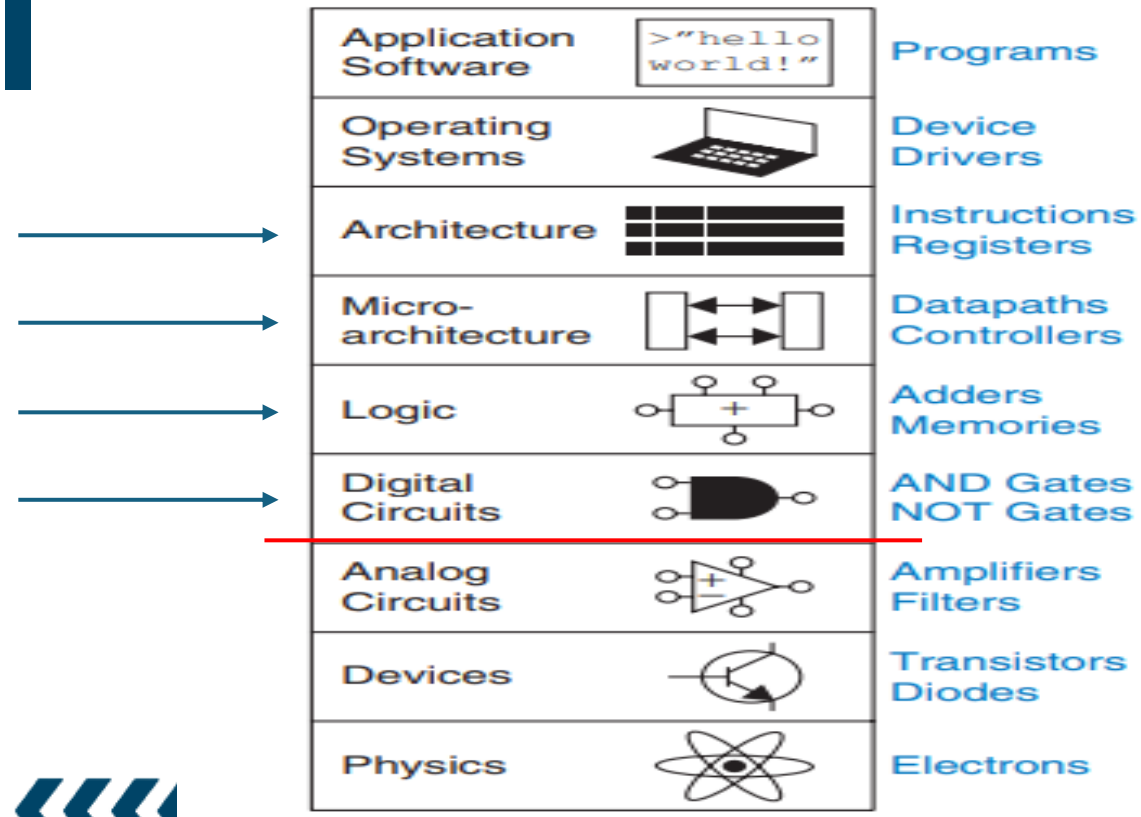
Abstraction in Digital Systems

Abstraction in Digital Systems



Abstraction in Digital Systems

- **Abstraction in Digital Systems**



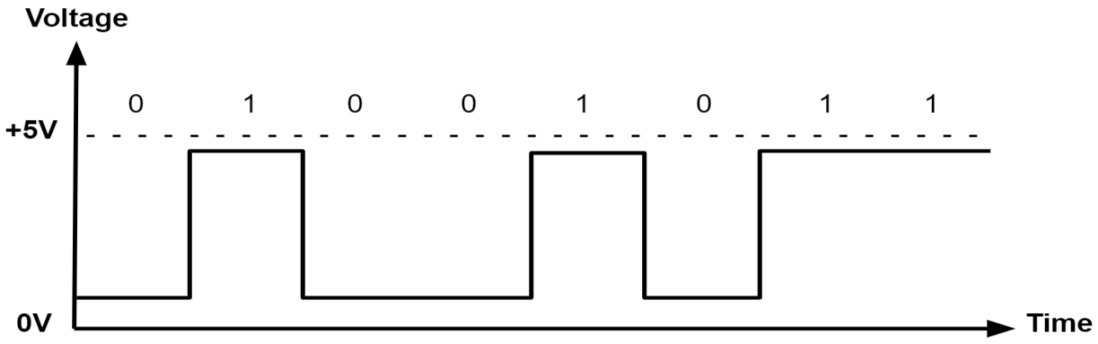
- We are interested in This Workshop and in Digital Design overall in this Layers Only.
- This raises a lot of Questions?!

Digital VS Analog

Analog Domain Vs Digital Domain

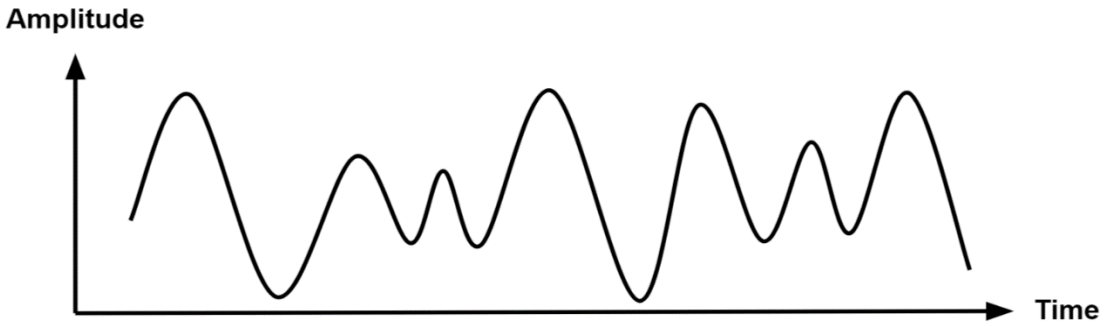
Digital

Signal is Discrete in Time and Value
”HIGH or LOW ”



Analog

Signal is Continuous in Time and Value



Advantages of Digital systems

Digital

- Immune to noise with easy regeneration and correction.
- Can be stored and encrypted.
- Can accurately perform any operation with relative cheap cost.

Analog

- **It's the natural language of the physical world.**
- **But!** it's hard to deal with data accurately due to noise.

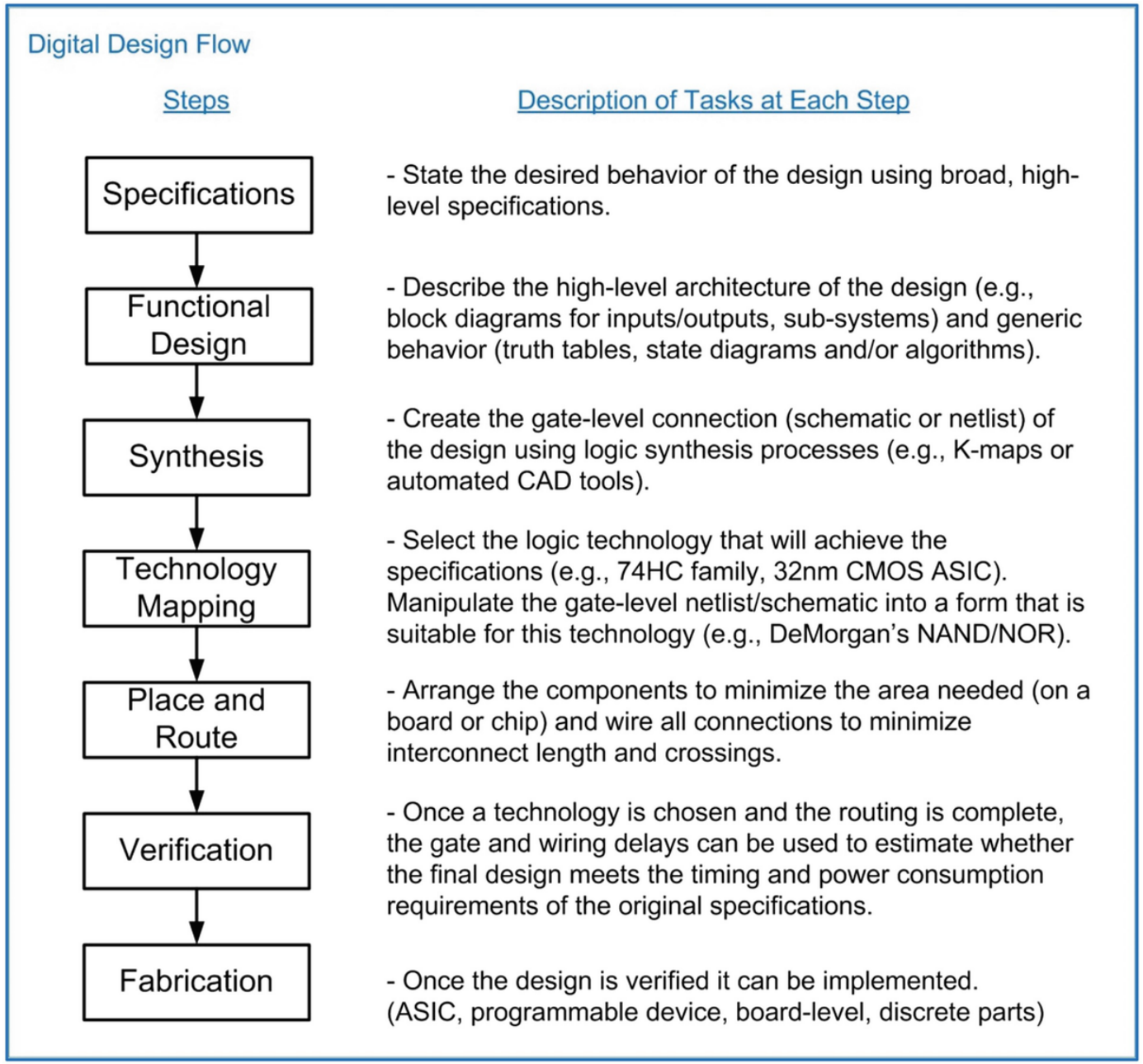
How Digital Engineers see Transistor "Heart of Electronics"

- In digital We don't care about its Design or Characteristics ; we just use it from pre-made Library contain all Logic gates and Circuits **"Boxes".**

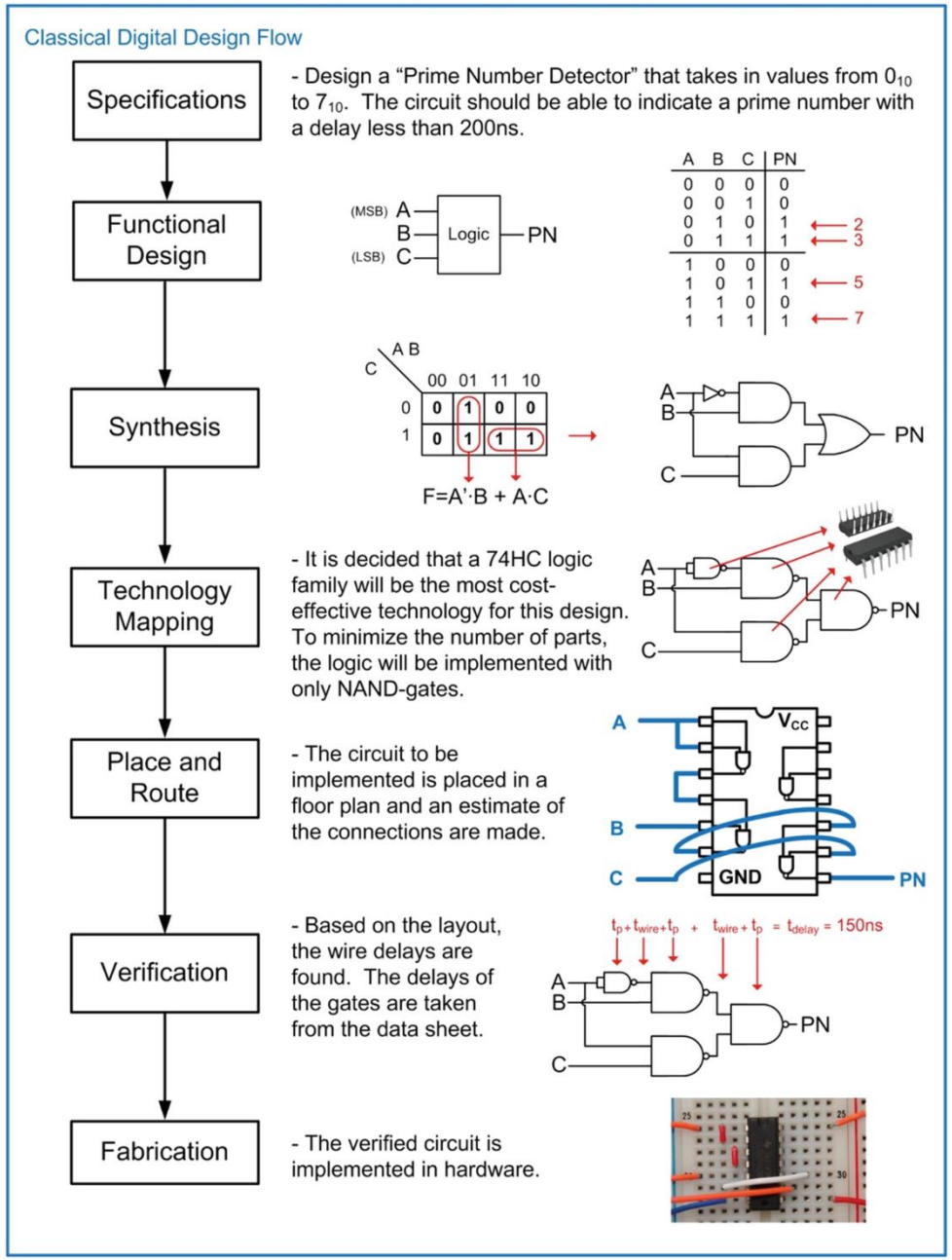
- So ,We don't design on Transistor Level unlike analog engineers.

Digital design flow

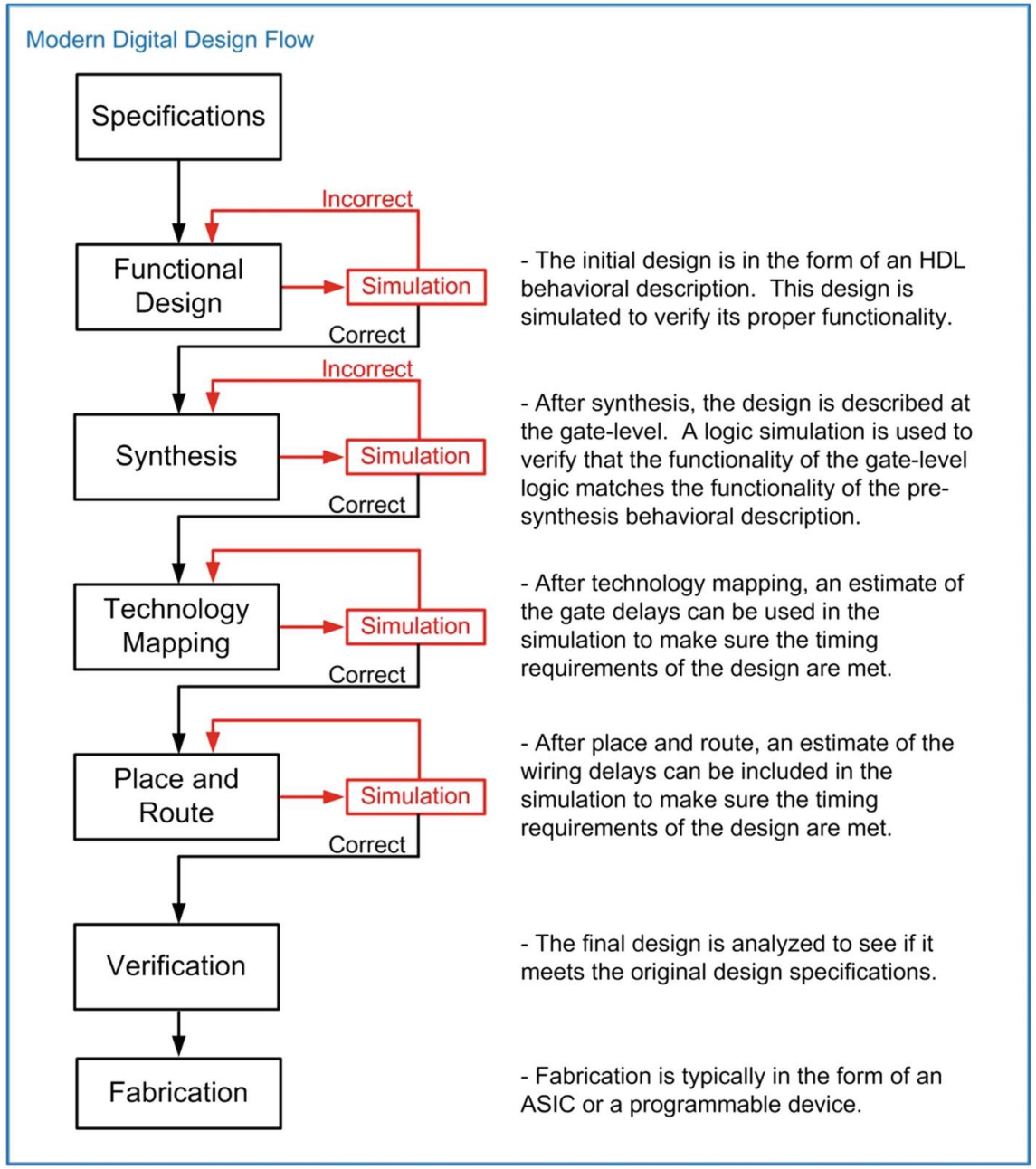
Digital generic design flow



Classic digital design flow

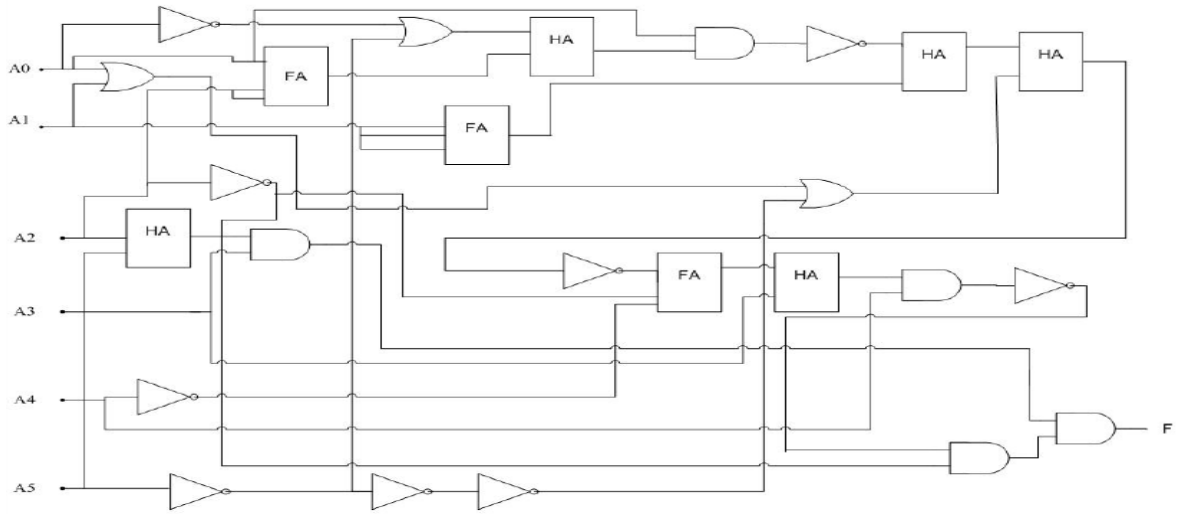


Modern digital design flow



History of HDLs and why high-level languages

- In the past, Digital designers used to implement their digital systems using traditional schematic schemes using CAD tools

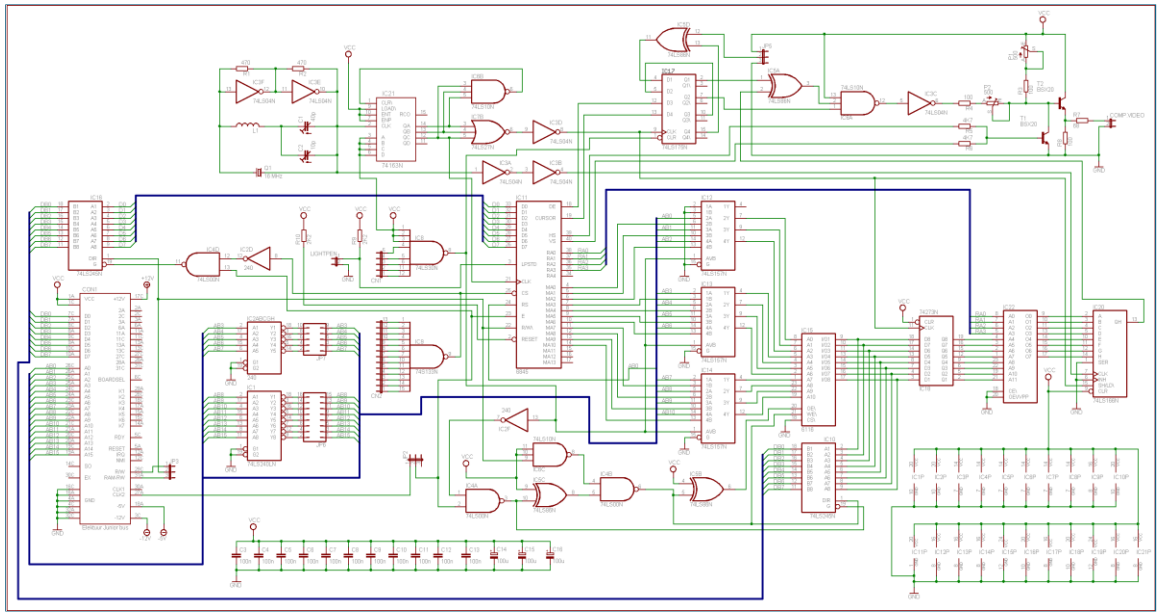


Is This Efficient ?

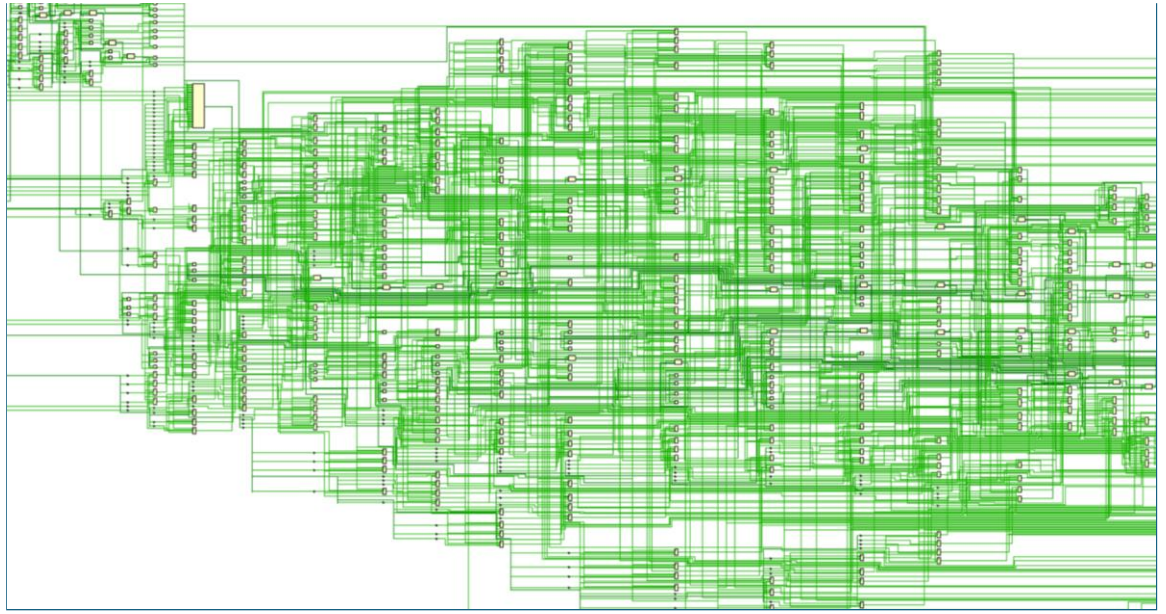
Of course not … lets see why

History of HDLs and why high-level languages

- Imagine having designs like those:



EASY !
(classic)



AND NOW !

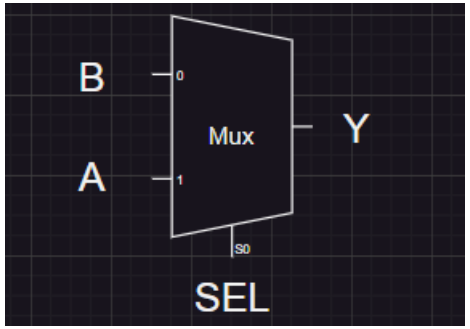
- So we turned to HDLs

History of HDLs and why high-level languages

- The most famous HDLs are :
  - ❖VHDL
  - ❖Verilog
  - ❖SystemVerilog
  - ❖SystemC
  - ❖MyHDL

Example:

```
if (SEL == 1)
    Y = A;
else
    Y = B;
```

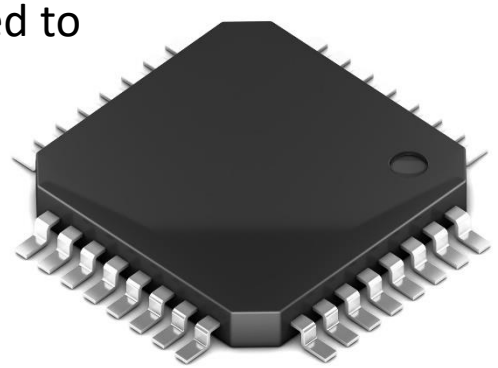


FPGA VS ASIC

FPGA VS ASIC

What is ASIC ?

- ASIC stands for Application-Specific Integrated Circuit.
- ASICs are custom-designed integrated circuits that are optimized to perform specific functions within electronic systems.
- They offer advantages such as optimized performance, lower per-unit costs for high-volume production, and customizability to meet the unique requirements of a particular application. However, ASIC development involves higher upfront costs and longer lead times compared to using off-the-shelf components or programmable devices like FPGAs.



FPGA VS ASIC

What is FPGA ?

- FPGA stands for Field-Programmable Gate Array. It's a type of integrated circuit (IC) that contains an array of programmable logic blocks (PLBs), configurable interconnects, and input/output blocks
- FPGAs are unique because they can be reprogrammed or reconfigured after manufacturing to implement various digital circuits and systems.

Why FPGA ?

- FPGAs offer designers a powerful platform for implementing custom digital circuits and systems, with the flexibility, reprogram ability, and performance necessary to meet the demands of a wide range of applications.
- It is mainly used for “prototyping” to reduce time to market

(prototyping: refers to the process of quickly implementing and testing a hardware design)

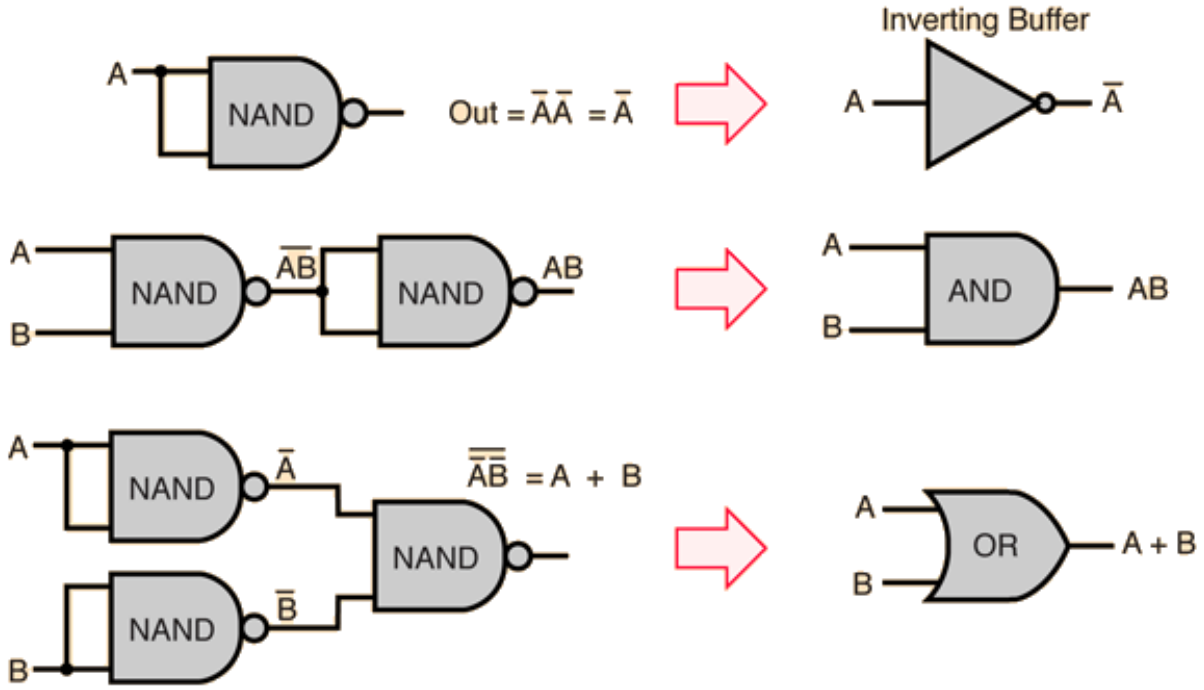
FPGA VS ASIC


FPGA
VS
ASIC
Comparison
Fast
Time to Market
Slow
Low
NRE
High
Simple
Design Flow
Complex
High
Unit Cost
Low
Medium
Performance
High
High
Power Consumption
Low
Medium
Unit Size
Low
HARDWAREBEE

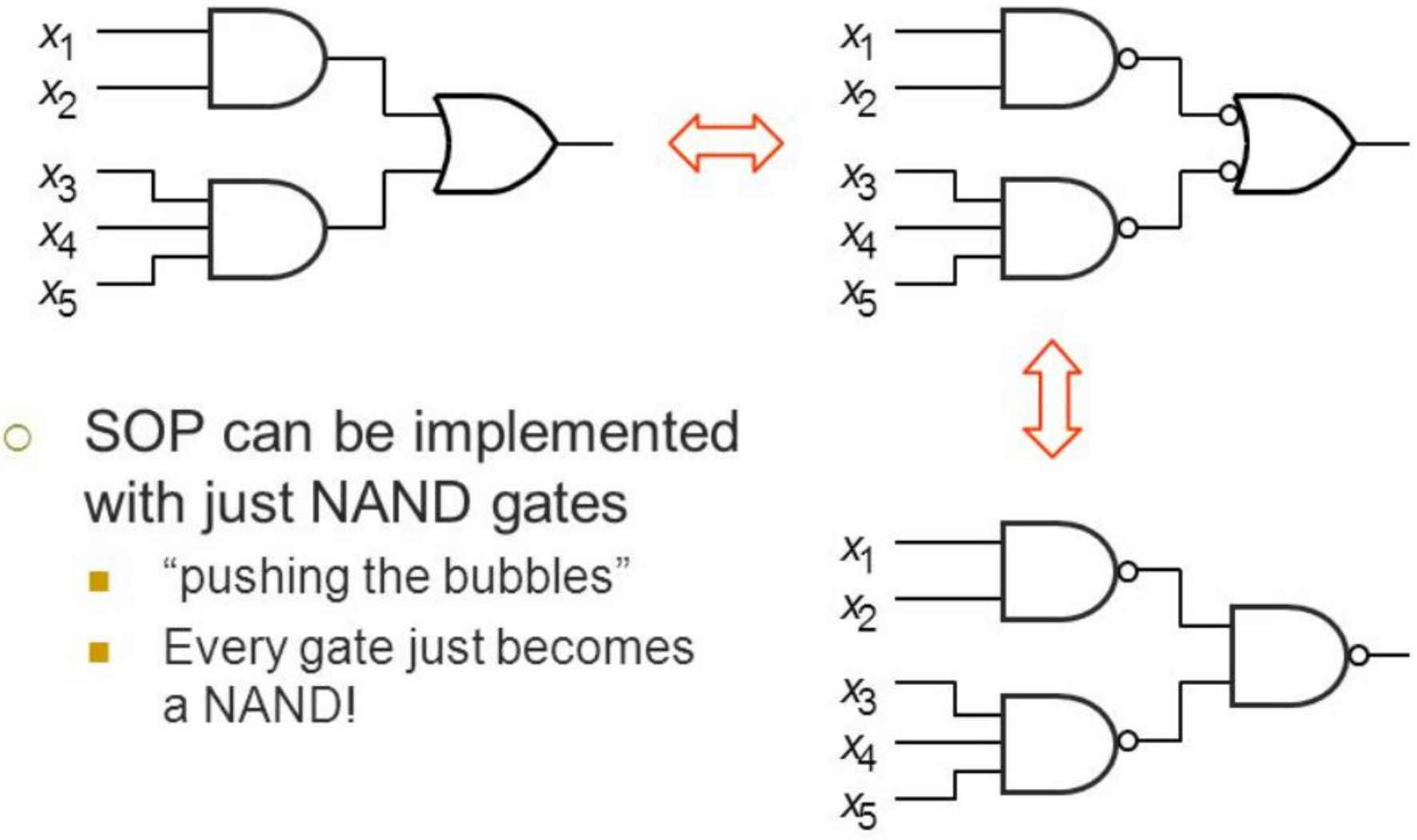
Gates and tech. mapping

Universal gates

- Definition:
  - Universal gates can implement any Boolean function without needing other gate types.
- Types of Universal Gates:
  - NAND Gate
  - NOR Gate

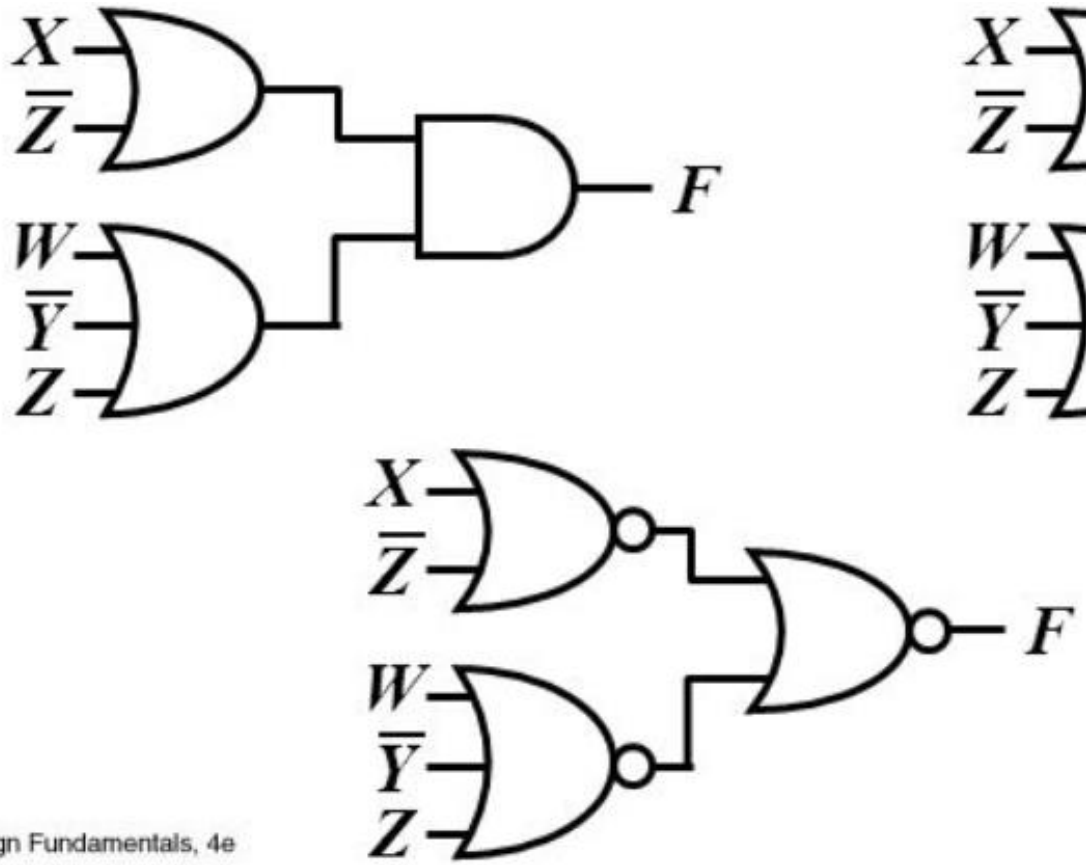


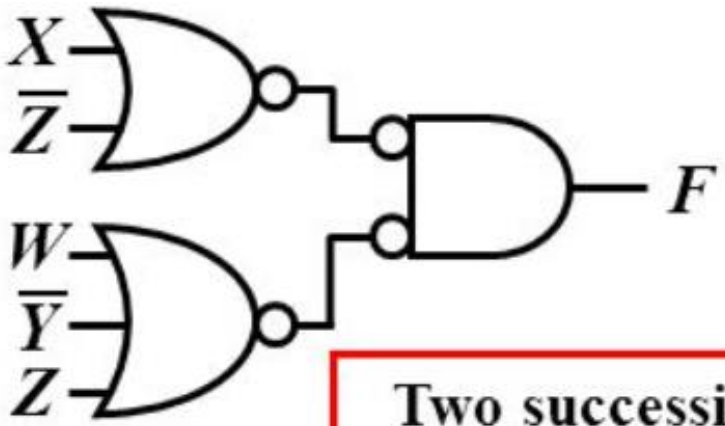
SOP using universal gates



- SOP can be implemented with just NAND gates
  - "pushing the bubbles"
  - Every gate just becomes a NAND!

POS using universal gates





Two successive bubbles on the same line cancel each other

Let’s make a design

Let's make a design

Classical flow

- Start with the specs : design a prime number detector circuit that detects the prime number of a 3 bit input

- Functional design

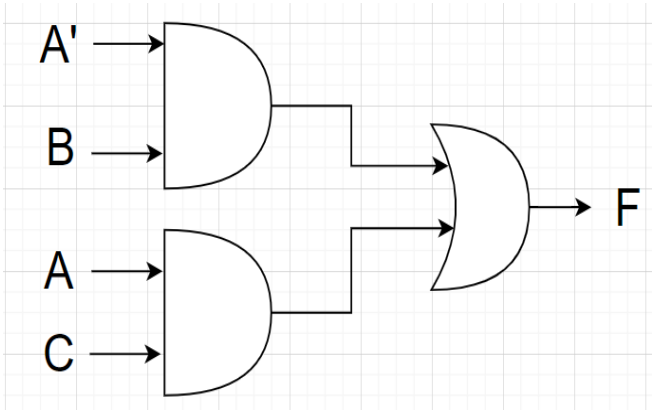
No.	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
9	1	1	0	0
7	1	1	1	1

KMAP

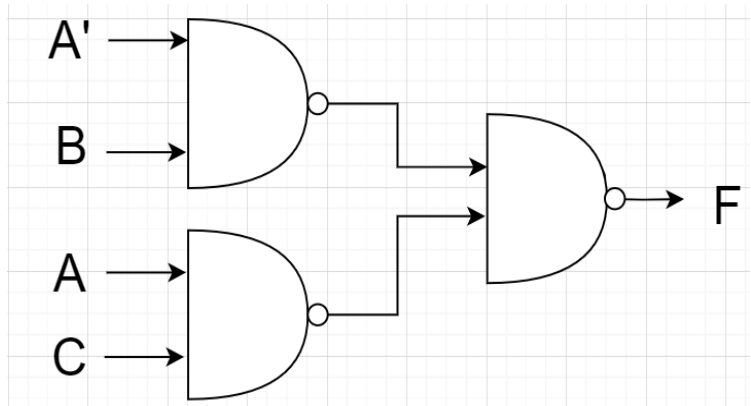
A\BC	00	01	11	10
0	0	0	1	1
1	0	1	1	0

F = A'B + AC

- Synthesis

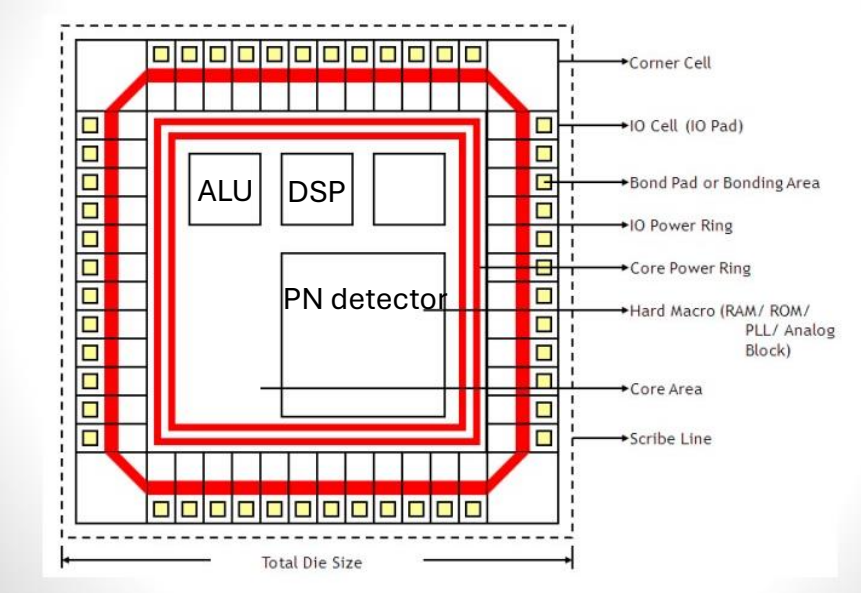


T.Mapping

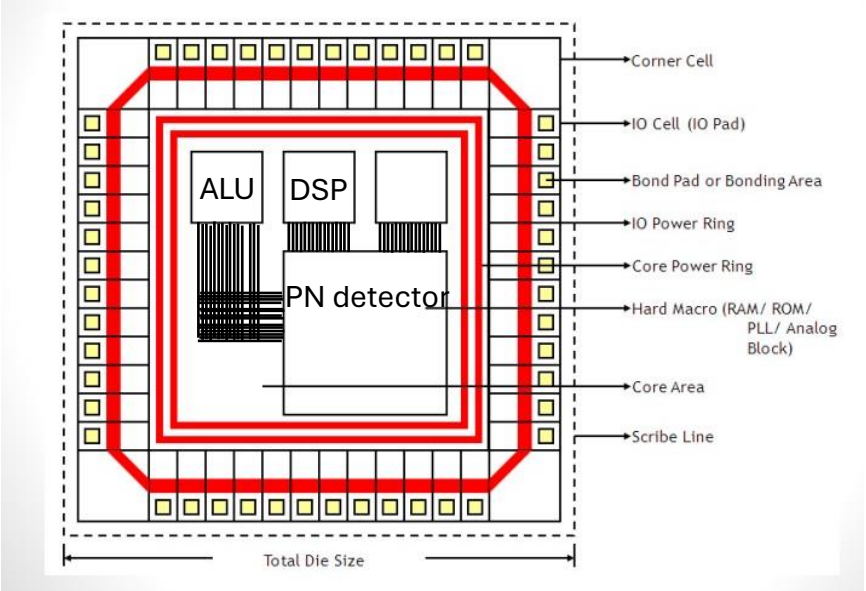


Let’s make a design

- Assurance of meeting timing constraints
- Floor planning



- Place and route



- GDSII
- Fabricate !!

Let’s make a design

Modern flow

- Start with the specs : design a prime number detector circuit that detects the prime number of a 3 bit input
- RTL design (Using Verilog)

```
module PN_detector(A,B,C,F);
input A,B,C;
output reg F;

always @ (*) begin
    if (({A,B,C} == 3'b010) || ({A,B,C} == 3'b011) || ({A,B,C} == 3'b101) || ({A,B,C} == 3'b111))
    F = 1;
    else
    F = 0;
end
endmodule
```

- Verification (using Verilog or SV)
- Synthesis (using EDA tools)
- Assurance of meeting timing constraints (using EDA tools)
- Floor planning
- Place and route
- GDSII
- Fabricate !!

IC industry in egypt

IC industry in Egypt

Questions?

Break

HDL ??

HDL

- HDL standing for hardware discerption language
- It is not a programming language
- You must design before coding think hardware remember functions example!
- If you start with coding, you will face two problems un synthesizable code or a very poor and bad design
- Two main languages are being used nowadays Verilog&VHDL
- Verilog will be studied in these sessions

Verilog basics

- Verilog is HDL (Hardware Discerption Language).
- It used to model and design digital circuits and systems.
- Similar syntax to C programming but it is not a programming language.

Verilog constructs

- The Verilog files have the extension .v
- Verilog is case sensitive
- Verilog line of code terminates with semicolon ‘;’
- Comments like in C by “//” for single line and “/*…..*/” for multiple lines
- The code here executes in parallel unlike the normal programming languages

Numerical Constructs

Numerical constants

- "X" is used by the simulators only to represent the wires that are not initialized by a known value
- "Z" is used for circuits not connected

High" logic one"	**1**
Low" logic zero"	0
Unknown	X
High impedance	Z

Numeric constants

- For couplies of wires (Bus), we can assign data to the whole bus without assigning wire by wire.

Syntax	Symbol	Numbering system
Bus = 100 or Bus=‘d100	d	Decimal (default)
Bus = ‘b0110_0100	b	Binary
Bus = ‘o144	o	Octal
Bus = ‘h64	h	Hexadecimal

General numbering representation

- [size]'[base][value]

Number	Stored value	Comment
5'b11010	11010	
5'b11_010	11010	_ignored
5'o32	11010	
5'h1a	11010	
5'd26	11010	
5'b0	00000	0 extended
5'b1	00001	0 extended

Operators

Operators

- Bitwise operators
  - Performs bit by bit evaluation between two operands
  - 2'b01 & 2'b11 = {0&1,1&1}
  - ~(3'b101) = 3'b010

Not	~A
And	A&B
Or	A|B
Xor	A^B
Xnor	~(A^B), (A~^B)

Operators

- Logical operators
  - Returns one-bit 1 or 0 (true or false)
  - Note: A&&B==1 if neither A or B ==0
  - !(2'b01) = 0 while ~(2'b01) = 2'b10
  - "==" tests logical equality for '1' and '0' only

    all other will result in 'X'
  - "===" tests 4-state logical equality (1,0,Z,X)

Not	!A
And	A&&B
Or	A||B
Logical Equality	A==B
Case Equality	A===B

Operators

- Arithmetic operators
  - Used to perform arithmetic operations

Addition	+
Subtraction	-
Multiplication	*
Division	/
Modulus	%

Operators

- Relational operators
  - Compares 2 operands and returns 1 or 0

Less than	<
Less than or equal	<=
Greater than	>
Greater than or equal	>=
Equal to	==

Operators

- Shift operators
  - Shift the first operand by the number specified by the second operand

Logic Shift	A>>1 shift right with 1bit A<<2 shift left with 2bits	8'b00010011>>1 =8'b00001001 8'b00010011<<2 =8'b01001100
Arithmetic Shift	A>>>1 A<<<2	8'b11110011>>>1 =8'b11111001 8'b00010011<<<2 =8'b01001100

Operators

- Concatenation and replication operators
  - {2'b00,2'b11,2'b00,2'b11} = 8'b00_11_00_11
  - {{4{2'b11}},8'b0} = 16'b1111_1111_0000_0000
  - Concatenation and replication have very useful applications and they are very fast and easy to use

Operators

- reduction operators
  - Evaluate bit-by-bit of a vector (unary operators)
  - &(4'b0111) = {0&1&1&1} = 0

And	&A
Or	|A
Xor	^A

Operators

- Conditional operator
  - Condition ? True : False
  - Any condition can be written and when its true the whole expression will be reduced to which written in true part
  - A= S? B:C if S is true then A=B else A=C

Operators

Verilog Operator	Name	Functional Group
[ ]	bit-select or part-select	
( )	parenthesis	
! ~ & | ~& ~| ^ ~^ or ^~	logical negation negation reduction AND reduction OR reduction NAND reduction NOR reduction XOR reduction XNOR	logical bit-wise reduction reduction reduction reduction reduction reduction
+ -	unary (sign) plus unary (sign) minus	arithmetic arithmetic
{ }	concatenation	concatenation
{{ }}	replication	replication
* / %	multiply divide modulus	arithmetic arithmetic arithmetic
+ -	binary plus binary minus	arithmetic arithmetic

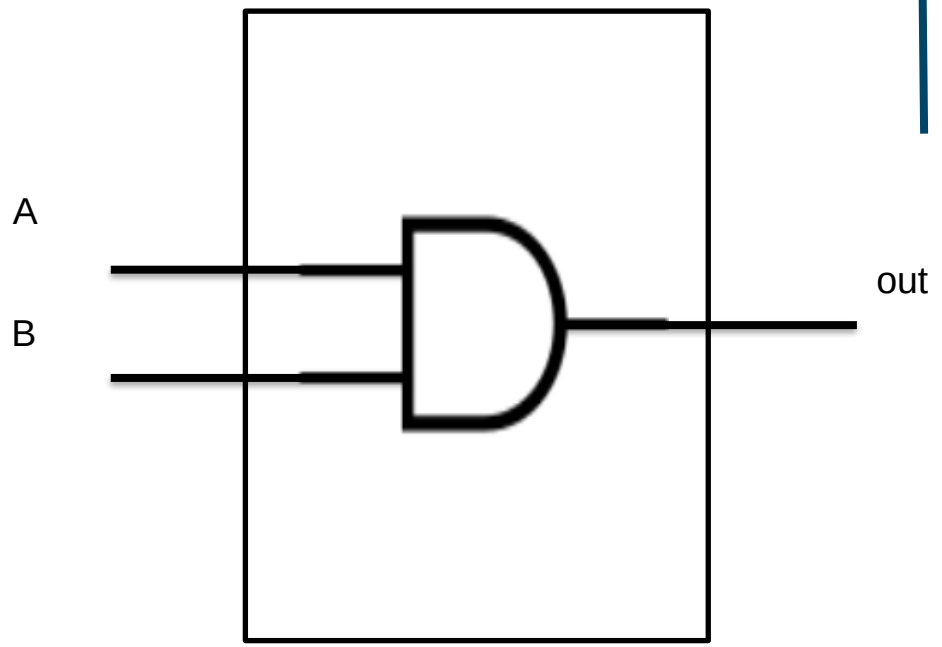
Operators

<< >>	shift left shift right	shift shift
> >= < <=	greater than greater than or equal to less than less than or equal to	relational relational relational relational
== !=	logical equality logical inequality	equality equality
=== !==	case equality case inequality	equality equality
&	bit-wise AND	bit-wise
^ ^~ or ~^	bit-wise XOR bit-wise XNOR	bit-wise bit-wise
|	bit-wise OR	bit-wise
&&	logical AND	logical
||	logical OR	logical
?:	conditional	conditional

Start coding with Verilog

Start coding with Verilog

```
1   module And_gate
2   (
3   input A,B,
4   output Out
5   );
6
7
8
9   assign Out=A&&B;
10
11  and(Out,A,B);
12
13  assign Out=A and B;
14
15
16  endmodule
17
```



Start coding with Verilog

- Module
  - Basic building block in Verilog
  - Each module has directional ports (input , output) used to communicate with the module
  - A module can be an element or a collection of lower-level design blocks
  - A module declared with keyword module
  - A corresponding keyword endmodule must appear at the end of the module
  - Ports are declared as wires by default

Start coding with Verilog

- wire
  - Wire represents a physical wire that is used inside a circuit or module
  - Wire doesn't store its value and so must be continuously driven
  - Bus is an array of wires moving together in parallel to transfer data
  - Note that we can type the width of the bus in the square brackets like this ex. [0:3] but here the MSB will be [0] and LSB will be [3] which is little bit desending order confusing, so we used to write it in this format [MSB:LSB]

```
wire A;                 // 1-bit wire
wire [31:0] W;          //32-bit bus
wire [3:0] B,C,D;       //3 (4-bit) busses
```

Start coding with Verilog

- reg
  - Reg used inside a sequential Circuits
  - Reg Storges the value until it updated
- Parameters
  - Parameters in Verilog are constants that allow you to create flexible and reusable modules. They define configurable values that can be changed when a module is instantiated, without modifying the original code.

```
1   module example_1
2   (
3
4   );
5
6   parameter width=8;
7
8   reg reg1;
9
10  reg [width-1:0] reg2;
11
12  wire [width-1:0] W;
13
14
15  endmodule
```

```
1   module example_1 #(parameter width=8)
2   (
3
4   );
5
6   reg reg1;
7
8   reg [width-1:0] reg2;
9
10  wire [width-1:0] W;
11
12
13  endmodule
```

Start coding with Verilog

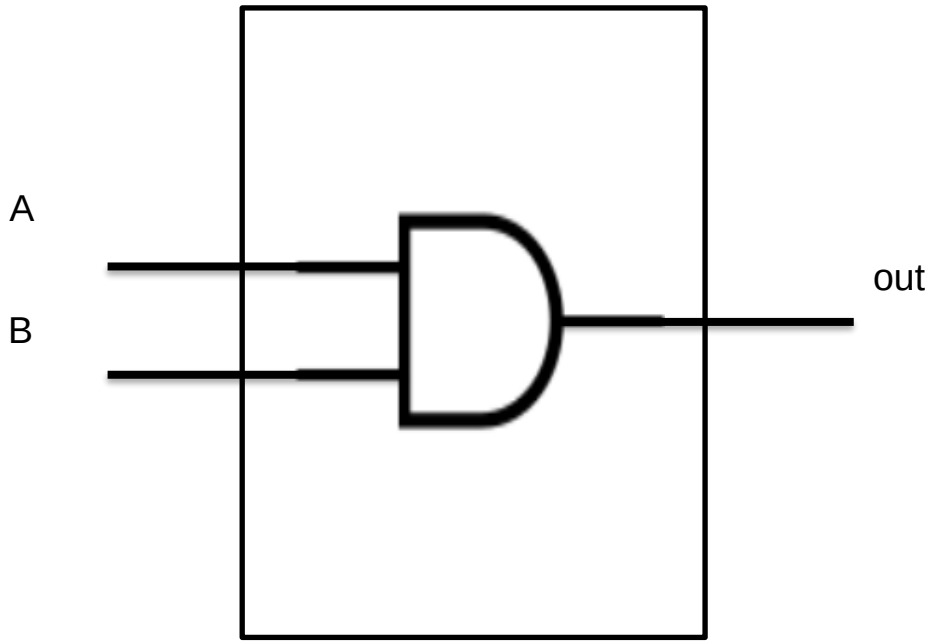
- Continuous assignment
  - Continuous assignment are used to describe a behavior equivalent to combinational logic
  - Keyword “assign” is used for the assignment of wires
  - Continuous assignments are continuously evaluated so when ever the RHS changes, the LFS is updated (typical combinational logic)

Lab 1

Implement the following and gate

input : A,B

output :out

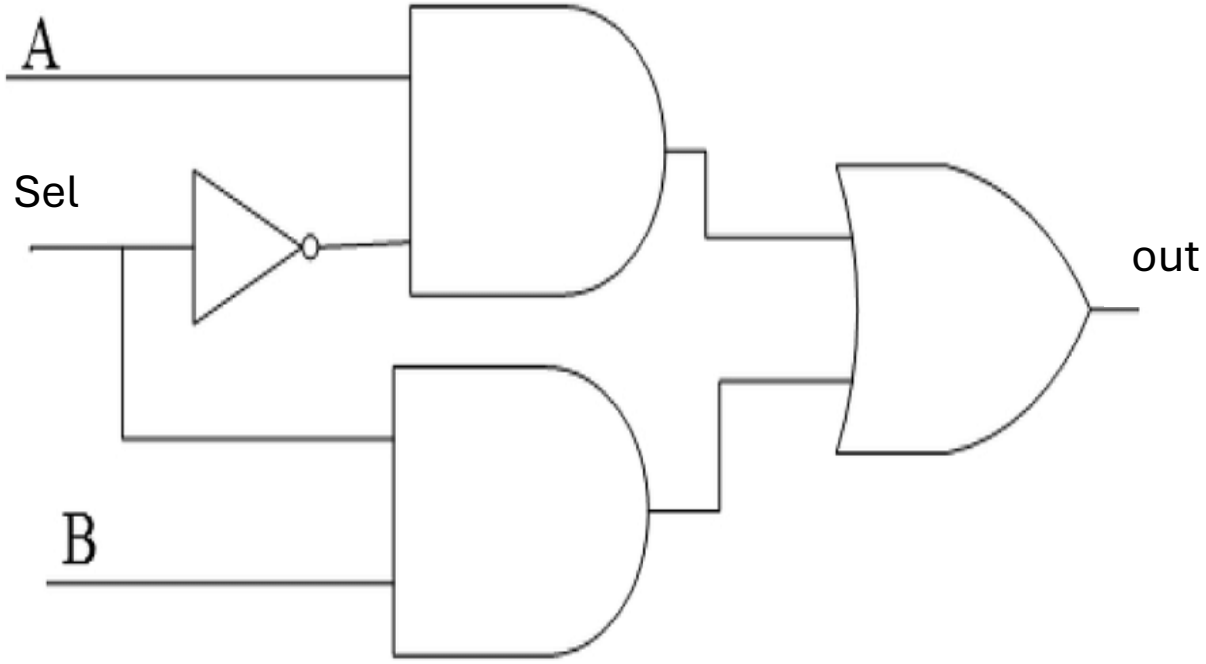


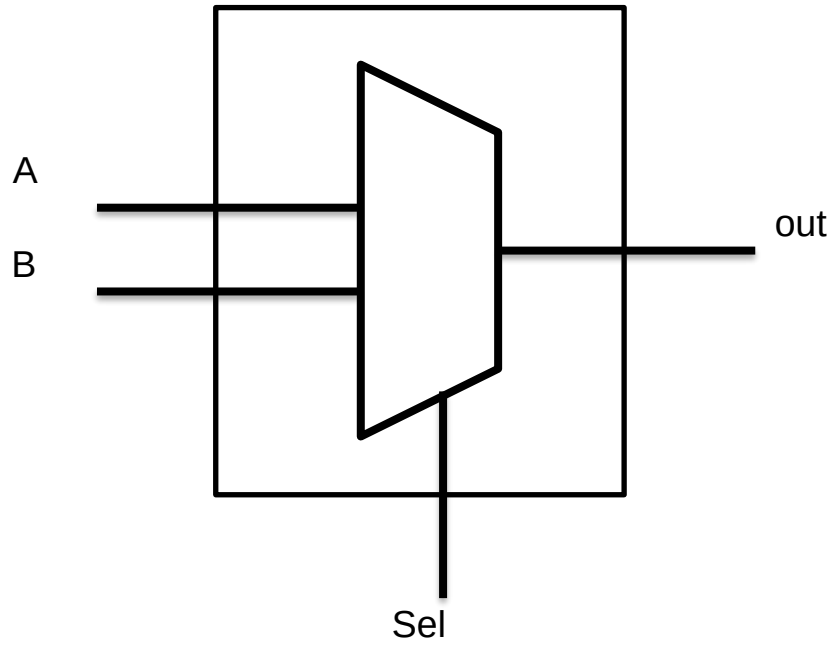
Lab 2

Implement a 2*1 mux in two different ways

Input: A, B, Sel

Output: out





Thank You